

Lab Report No 4
Understanding and implementing KNN algorithm
for classification problem
Artificial Intelligence



Submitted By:

[Huzaifa Shahbaz]

[21-CE-009]

[Aqsa Shah]

[21-CE-039]

Submitted to:

Engr. Hareem Khan

Dated:

18th Mar, 2025

Department of Computer Engineering,
HITEC University, Taxila

Lab Example No 1:

Solution:

Brief description (3-5 lines)
IRIS DATASETS:
A classification problem solved by using KNN and implemented for IRIS dataset.
The code
<pre>import numpy as np import matplotlib.pyplot as plt from matplotlib.colors import ListedColormap from scipy.stats import mode from sklearn.metrics import accuracy_score, confusion_matrix from sklearn.datasets import load_iris from numpy.random import randint # Function Definitions # Euclidean Distance Function def euclidean(p1, p2): return np.sqrt(np.sum((p1 - p2) ** 2)) # KNN Prediction Function def predict(x_train, y_train, x_input, k): op_labels = [] for item in x_input: point_dist = [(euclidean(x_train[j], item), y_train[j]) for j in range(len(x_train))] point_dist.sort(key=lambda x: x[0]) # Sort by distance k_nearest_labels = [label for _, label in point_dist[:k]] most_common = mode(k_nearest_labels, keepdims=True) # Majority voting op_labels.append(most_common.mode[0]) return np.array(op_labels) # Load Dataset iris = load_iris() X = iris.data y = iris.target feature_names = iris.feature_names target_names = iris.target_names # Train-Test Split np.random.seed(42) # For reproducibility train_indices = randint(0, 150, 100) test_indices = np.array([i for i in range(150) if i not in train_indices])[50] X_train, y_train = X[train_indices], y[train_indices] X_test, y_test = X[test_indices], y[test_indices]</pre>

```

# Apply KNN
k = 7 # Number of neighbors
y_pred = predict(X_train, y_train, X_test, k)
# Compute Accuracy
accuracy = accuracy_score(y_test, y_pred)
print(f'Accuracy: {accuracy:.4f}')
# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:")
print(cm)
# Data Visualization
# Select only two features for visualization (Sepal Length & Sepal Width)
feature_idx = [0, 1]
X_train_2d, X_test_2d = X_train[:, feature_idx], X_test[:, feature_idx]
# Create Mesh Grid for Decision Boundary
h = 0.02 # Step size
x_min, x_max = X[:, feature_idx[0]].min() - 1, X[:, feature_idx[0]].max() + 1
y_min, y_max = X[:, feature_idx[1]].min() - 1, X[:, feature_idx[1]].max() + 1
xx, yy = np.meshgrid(np.arange(x_min, x_max, h), np.arange(y_min, y_max, h))
# Predict for Each Grid Point
Z = predict(X_train_2d, y_train, np.c_[xx.ravel(), yy.ravel()], k)
Z = Z.reshape(xx.shape)
# Define Color Maps
cmap_light = ListedColormap(['#FFAAAA', '#AAFFAA', '#AAAAFF'])
cmap_bold = ListedColormap(['#FF0000', '#00FF00', '#0000FF'])
# Plot Figures
plt.figure(figsize=(12, 10))
# 1. KNN Decision Boundary
plt.subplot(2, 2, 1)
plt.contourf(xx, yy, Z, alpha=0.4, cmap=cmap_light)
plt.scatter(X_train_2d[:, 0], X_train_2d[:, 1], c=y_train, cmap=cmap_bold,
            edgecolor='k', s=40)
plt.scatter(X_test_2d[:, 0], X_test_2d[:, 1], c=y_test, cmap=cmap_bold,
            marker='*', s=100, edgecolor='k')
plt.xlabel(feature_names[feature_idx[0]])
plt.ylabel(feature_names[feature_idx[1]])
plt.title(f'KNN Decision Boundary (k={k})')
plt.legend(['Training data', 'Test data'])
# 2. Iris Dataset Distribution (Sepal Length vs Sepal Width)
plt.subplot(2, 2, 2)

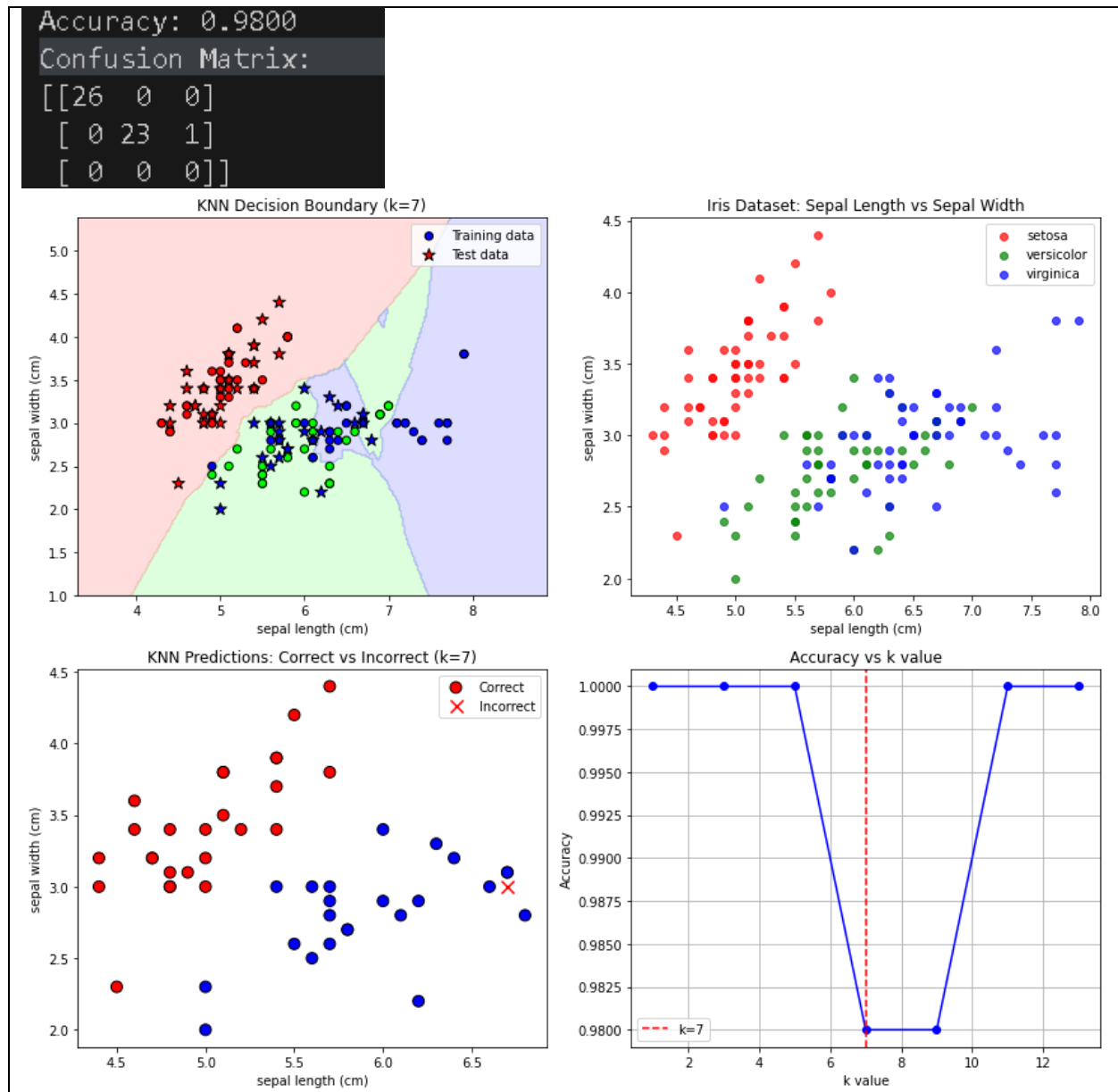
```

```

for i, color in zip(range(3), ['red', 'green', 'blue']):
    plt.scatter(X[y == i, 0], X[y == i, 1], color=color, alpha=0.7,
label=target_names[i])
plt.xlabel(feature_names[0])
plt.ylabel(feature_names[1])
plt.title('Iris Dataset: Sepal Length vs Sepal Width')
plt.legend()
# 3. Correct vs Incorrect Predictions
plt.subplot(2, 2, 3)
correct = y_pred == y_test
plt.scatter(X_test_2d[correct, 0], X_test_2d[correct, 1], c=y_test[correct],
cmap=cmap_bold, marker='o', s=80, edgecolor='k', label='Correct')
plt.scatter(X_test_2d[~correct, 0], X_test_2d[~correct, 1], c=y_test[~correct],
cmap=cmap_bold, marker='x', s=100, edgecolor='k', label='Incorrect')
plt.xlabel(feature_names[feature_idx[0]])
plt.ylabel(feature_names[feature_idx[1]])
plt.title(f'KNN Predictions: Correct vs Incorrect (k={k})')
plt.legend()
# 4. Accuracy vs k values
plt.subplot(2, 2, 4)
k_values = range(1, 15, 2)
accuracy_values = [accuracy_score(y_test, predict(X_train, y_train, X_test,
k_val)) for k_val in k_values]
plt.plot(k_values, accuracy_values, 'o-', color='blue')
plt.axvline(x=k, color='red', linestyle='--', label=f'k={k}')
plt.xlabel('k value')
plt.ylabel('Accuracy')
plt.title('Accuracy vs k value')
plt.grid(True)
plt.legend()
plt.tight_layout()
plt.show()
# Print Accuracy for Different k Values
print("\nAccuracy for different k values:")
for k_val, acc in zip(k_values, accuracy_values):
    print(f'k = {k_val}: {acc:.4f}')

```

The results (Screenshot)



Lab Task No 1:

Solution:

Brief description (3-5 lines)

Write the above-mentioned program for different datasets. Show data head with features used and corresponding labels in case of classification problem. Evaluate results for all metrics.

The code

```
import numpy as np
```

```

import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris, load_wine, load_breast_cancer,
fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score,
f1_score, roc_auc_score, confusion_matrix
from sklearn.metrics import classification_report
# Set random seed for reproducibility
np.random.seed(42)
# Function to load and preprocess classification datasets
def load_classification_dataset(dataset_name):
    if dataset_name == 'iris':
        data = load_iris()
        X = pd.DataFrame(data.data, columns=data.feature_names)
        y = pd.Series(data.target, name='target')
        class_names = data.target_names
    elif dataset_name == 'wine':
        data = load_wine()
        X = pd.DataFrame(data.data, columns=data.feature_names)
        y = pd.Series(data.target, name='target')
        class_names = data.target_names
    elif dataset_name == 'breast_cancer':
        data = load_breast_cancer()
        X = pd.DataFrame(data.data, columns=data.feature_names)
        y = pd.Series(data.target, name='target')
        class_names = data.target_names
    return X, y, class_names
# Function to evaluate classifiers and collect metrics
def evaluate_classifier(X, y, class_names, dataset_name):
    # Split data
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
random_state=42)
    # Standardize features
    scaler = StandardScaler()
    X_train_scaled = scaler.fit_transform(X_train)
    X_test_scaled = scaler.transform(X_test)

```

```

# Initialize models
models = {
    'Logistic Regression': LogisticRegression(max_iter=1000,
random_state=42),
    'Random Forest': RandomForestClassifier(n_estimators=100,
random_state=42)
}
results = {}
# Train and evaluate each model
for name, model in models.items():
    model.fit(X_train_scaled, y_train)
    y_pred = model.predict(X_test_scaled)
    # For multi-class, use 'macro' averaging
    avg_method = 'macro' if len(class_names) > 2 else 'binary'
    # Calculate metrics
    metrics = {
        'accuracy': accuracy_score(y_test, y_pred),
        'precision': precision_score(y_test, y_pred, average=avg_method),
        'recall': recall_score(y_test, y_pred, average=avg_method),
        'f1': f1_score(y_test, y_pred, average=avg_method)
    }
    # Add ROC AUC for binary classification
    if len(class_names) == 2:
        y_prob = model.predict_proba(X_test_scaled)[: , 1]
        metrics['roc_auc'] = roc_auc_score(y_test, y_prob)
    # Add confusion matrix
    metrics['confusion_matrix'] = confusion_matrix(y_test, y_pred)
    # Add classification report
    metrics['classification_report'] = classification_report(y_test, y_pred,
target_names=class_names)
    results[name] = metrics
return results
# Main execution
def main():
    # List of datasets to analyze
    datasets = ['iris', 'wine', 'breast_cancer']
    all_results = {}
    # Process each dataset
    for dataset_name in datasets:
        print(f'\n{'='*50}')

```

```

print(f'Dataset: {dataset_name.upper()}')
print(f'{'='*50}')
# Load and prepare data
X, y, class_names = load_classification_dataset(dataset_name)
# Display data info
print(f'\nData shape: {X.shape}')
print(f'Number of classes: {len(class_names)}')
print(f'Class names: {class_names}')
print(f'Class distribution: {np.bincount(y)}')
# Display data head
print("\nFeatures (first 5 rows):")
print(X.head())
print("\nLabels (first 5 values):")
print(y.head())
# Train and evaluate models
results = evaluate_classifier(X, y, class_names, dataset_name)
all_results[dataset_name] = results
# Print results
for model_name, metrics in results.items():
    print(f'\nModel: {model_name}')
    print(f'Accuracy: {metrics['accuracy']:.4f}')
    print(f'Precision: {metrics['precision']:.4f}')
    print(f'Recall: {metrics['recall']:.4f}')
    print(f'F1 Score: {metrics['f1']:.4f}')
    if 'roc_auc' in metrics:
        print(f'ROC AUC: {metrics['roc_auc']:.4f}')
    print("\nConfusion Matrix:")
    print(metrics['confusion_matrix'])
    print("\nClassification Report:")
    print(metrics['classification_report'])
# Visualize results
plot_comparison(all_results)
# Function to create comparison plots
def plot_comparison(all_results):
    metrics_to_plot = ['accuracy', 'precision', 'recall', 'f1']
    datasets = list(all_results.keys())
    models = list(all_results[datasets[0]].keys())
    # Create a figure with subplots
    fig, axes = plt.subplots(len(metrics_to_plot), 1, figsize=(12, 16))
    # Set spacing between subplots

```



```

plt.subplots_adjust(hspace=0.4)
# Plot each metric
for i, metric in enumerate(metrics_to_plot):
    ax = axes[i]
    # Prepare data for plotting
    x = np.arange(len(datasets))
    width = 0.35
    # Plot bars for each model
    for j, model in enumerate(models):
        values = [all_results[dataset][model][metric] for dataset in datasets]
        ax.bar(x + j*width, values, width, label=model)
    # Customize the plot
    ax.set_title(f'{metric.capitalize()} Comparison')
    ax.set_xlabel('Dataset')
    ax.set_ylabel(metric.capitalize())
    ax.set_xticks(x + width/2)
    ax.set_xticklabels(datasets)
    ax.legend()
    # Add values on top of bars
    for j, model in enumerate(models):
        values = [all_results[dataset][model][metric] for dataset in datasets]
        for k, v in enumerate(values):
            ax.text(x[k] + j*width, v + 0.01, f'{v:.3f}', ha='center', va='bottom',
fontsize=8)
    plt.suptitle('Performance Metrics Comparison Across Datasets', fontsize=16)
    plt.tight_layout(rect=[0, 0, 1, 0.97])
    plt.show()
# Run the program
if __name__ == "__main__":
    main()

```

The results (Screenshot)

```
=====
Dataset: IRIS
=====
```

```
Data shape: (150, 4)
Number of classes: 3
Class names: ['setosa' 'versicolor' 'virginica']
Class distribution: [50 50 50]
```

```
Features (first 5 rows):
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)
0	5.1	3.5	1.4	0.2
1	4.9	3.0	1.4	0.2
2	4.7	3.2	1.3	0.2
3	4.6	3.1	1.5	0.2
4	5.0	3.6	1.4	0.2

```
Labels (first 5 values):
```

```
0    0
```

```
1    0
```

```
2    0
```

```
3    0
```

```
4    0
```

```
Name: target, dtype: int32
```

```
Model: Logistic Regression
```

```
Accuracy: 1.0000
```

```
Precision: 1.0000
```

```
Recall: 1.0000
```

```
F1 Score: 1.0000
```

```
Confusion Matrix:
```

```
[[19  0  0]
```

```
 [ 0 13  0]
```

```
 [ 0  0 13]]
```

```
Classification Report:
```

	precision	recall	f1-score	support
setosa	1.00	1.00	1.00	19
versicolor	1.00	1.00	1.00	13
virginica	1.00	1.00	1.00	13

```

    accuracy          1.00          45
  macro avg          1.00          45
weighted avg          1.00          45
```

Model: Random Forest

Accuracy: 1.0000

Precision: 1.0000

Recall: 1.0000

F1 Score: 1.0000

Confusion Matrix:

```
[[19  0  0]
 [ 0 13  0]
 [ 0  0 13]]
```

Classification Report:

	precision	recall	f1-score	support
setosa	1.00	1.00	1.00	19
versicolor	1.00	1.00	1.00	13
virginica	1.00	1.00	1.00	13
accuracy			1.00	45
macro avg	1.00	1.00	1.00	45
weighted avg	1.00	1.00	1.00	45

```
=====
Dataset: WINE
=====
```

```
Data shape: (178, 13)
Number of classes: 3
Class names: ['class_0' 'class_1' 'class_2']
Class distribution: [59 71 48]
```

```
Features (first 5 rows):
```

	alcohol	malic_acid	ash	...	hue	od280/od315_of_diluted_wines	proline
0	14.23	1.71	2.43	...	1.04	3.92	1065.0
1	13.20	1.78	2.14	...	1.05	3.40	1050.0
2	13.16	2.36	2.67	...	1.03	3.17	1185.0
3	14.37	1.95	2.50	...	0.86	3.45	1480.0
4	13.24	2.59	2.87	...	1.04	2.93	735.0

```
[5 rows x 13 columns]
```

```
Labels (first 5 values):
```

```
0    0
1    0
2    0
3    0
4    0
```

```
Name: target, dtype: int32
```

```
Model: Logistic Regression
Accuracy: 0.9815
Precision: 0.9778
Recall: 0.9841
F1 Score: 0.9804
```

```
Confusion Matrix:
[[19  0  0]
 [ 0 20  1]
 [ 0  0 14]]
```

```
Classification Report:
              precision    recall  f1-score   support

   class_0       1.00      1.00      1.00        19
   class_1       1.00      0.95      0.98        21
   class_2       0.93      1.00      0.97        14

 accuracy              0.98              0.98              0.98              54
 macro avg              0.98              0.98              0.98              54
weighted avg              0.98              0.98              0.98              54
```

```
Model: Random Forest
Accuracy: 1.0000
```

```
Precision: 1.0000
Recall: 1.0000
F1 Score: 1.0000
```

```
Confusion Matrix:
```

```
[[19  0  0]
 [ 0 21  0]
 [ 0  0 14]]
```

```
Classification Report:
```

	precision	recall	f1-score	support
class_0	1.00	1.00	1.00	19
class_1	1.00	1.00	1.00	21
class_2	1.00	1.00	1.00	14
accuracy			1.00	54
macro avg	1.00	1.00	1.00	54
weighted avg	1.00	1.00	1.00	54

```
=====
Dataset: BREAST_CANCER
=====
```

```
Data shape: (569, 30)
Number of classes: 2
```

```
Class names: ['malignant' 'benign']
Class distribution: [212 357]
```

```
Features (first 5 rows):
```

	mean radius	mean texture	...	worst symmetry	worst fractal dimension
0	17.99	10.38	...	0.4601	0.11890
1	20.57	17.77	...	0.2750	0.08902
2	19.69	21.25	...	0.3613	0.08758
3	11.42	20.38	...	0.6638	0.17300
4	20.29	14.34	...	0.2364	0.07678

```
[5 rows x 30 columns]
```

```
Labels (first 5 values):
```

0	0
1	0
2	0
3	0
4	0

```
Name: target, dtype: int32
```

```
Model: Logistic Regression
```

```
Accuracy: 0.9825
```

```
Precision: 0.9907
```

```
Recall: 0.9815
```

```
F1 Score: 0.9860
```

```
ROC AUC: 0.9979
```


Confusion Matrix:

```
[[ 62   1]
 [   2 106]]
```

Classification Report:

	precision	recall	f1-score	support
malignant	0.97	0.98	0.98	63
benign	0.99	0.98	0.99	108
accuracy			0.98	171
macro avg	0.98	0.98	0.98	171
weighted avg	0.98	0.98	0.98	171

Model: Random Forest

Accuracy: 0.9708

Precision: 0.9640

Recall: 0.9907

F1 Score: 0.9772

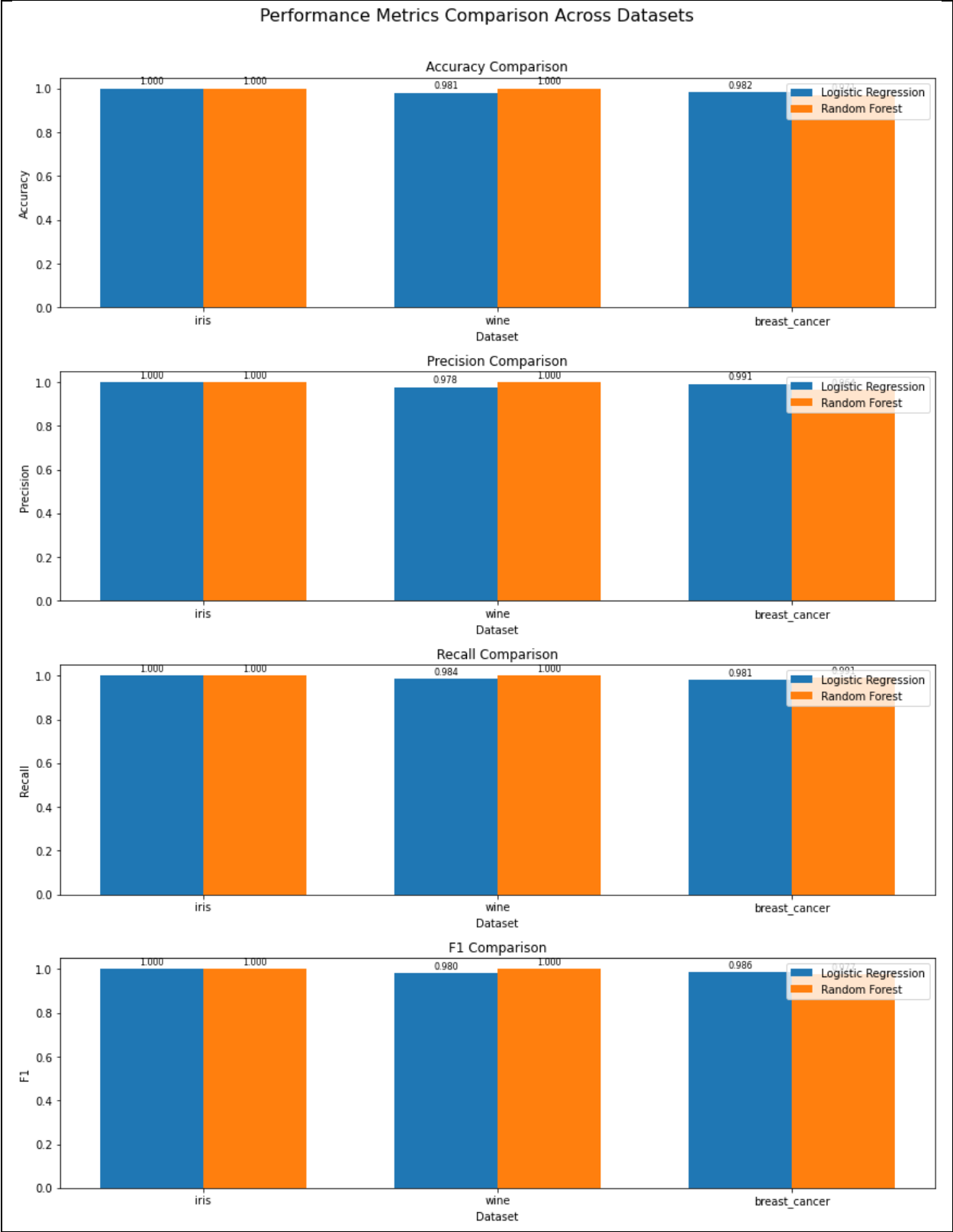
ROC AUC: 0.9969

Confusion Matrix:

```
[[ 59   4]
 [   1 107]]
```

Classification Report:

	precision	recall	f1-score	support
malignant	0.98	0.94	0.96	63
benign	0.96	0.99	0.98	108
Classification Report:				
	precision	recall	f1-score	support
malignant	0.98	0.94	0.96	63
benign	0.96	0.99	0.98	108
accuracy			0.97	171
macro avg	0.97	0.96	0.97	171
accuracy			0.97	171
macro avg	0.97	0.96	0.97	171
macro avg	0.97	0.96	0.97	171
weighted avg	0.97	0.97	0.97	171



Lab Task No 2:

Solution:

Brief description (3-5 lines)

Use KNN for regression dataset. Show data head with features used and corresponding labels in case of classification problem. Evaluate results for all metrics.

The code

<pre>import numpy as np import pandas as pd import matplotlib.pyplot as plt from sklearn.datasets import fetch_california_housing from sklearn.model_selection import train_test_split from sklearn.neighbors import KNeighborsRegressor from sklearn.preprocessing import StandardScaler from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score from sklearn.model_selection import GridSearchCV # Load the California housing dataset as an example regression dataset housing = fetch_california_housing() X = pd.DataFrame(housing.data, columns=housing.feature_names) y = housing.target # Display information about the dataset print("Dataset Information:") print(f'Number of samples: {X.shape[0]}') print(f'Number of features: {X.shape[1]}') print("\nFeature names:") print(housing.feature_names) print("\nTarget description:") print(housing.DESCR.split('\n')[4]) # Extract target description from dataset description # Show the first 5 rows of the dataset with features and target data_with_target = X.copy() data_with_target['Target (MedHouseValue)'] = y print("\nFirst 5 rows of the dataset:") print(data_with_target.head()) # Split the data into training and testing sets X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random state=42)</pre>
--

```

# Scale the features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
# Find the optimal value of K using GridSearchCV
param_grid = {'n_neighbors': np.arange(3, 21, 2)}
knn_grid = GridSearchCV(KNeighborsRegressor(), param_grid, cv=5,
scoring='neg_mean_squared_error')
knn_grid.fit(X_train_scaled, y_train)
# Print the best parameter
best_k = knn_grid.best_params_['n_neighbors']
print(f"\nBest value of K: {best_k}")
# Train KNN regression model with the best K
knn_model = KNeighborsRegressor(n_neighbors=best_k)
knn_model.fit(X_train_scaled, y_train)
# Make predictions
y_pred = knn_model.predict(X_test_scaled)
# Evaluate the model using various metrics
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
mae = mean_absolute_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)
print("\nModel Evaluation Metrics:")
print(f'Mean Squared Error (MSE): {mse:.4f}')
print(f'Root Mean Squared Error (RMSE): {rmse:.4f}')
print(f'Mean Absolute Error (MAE): {mae:.4f}')
print(f'R-squared (R²): {r2:.4f}')
# Calculate additional metrics
explained_variance = 1 - (np.var(y_test - y_pred) / np.var(y_test))
mape = np.mean(np.abs((y_test - y_pred) / y_test)) * 100
print(f'Explained Variance: {explained_variance:.4f}')
print(f'Mean Absolute Percentage Error (MAPE): {mape:.4f}%')
# Visualize actual vs predicted values
plt.figure(figsize=(10, 6))
plt.scatter(y_test, y_pred, alpha=0.5)
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--')
plt.xlabel('Actual Values')
plt.ylabel('Predicted Values')
plt.title('KNN Regression: Actual vs Predicted Values')
plt.tight_layout()

```

```

plt.show()
# Visualize error distribution
errors = y_test - y_pred
plt.figure(figsize=(10, 6))
plt.hist(errors, bins=30)
plt.xlabel('Prediction Error')
plt.ylabel('Count')
plt.title('Distribution of Prediction Errors')
plt.axvline(x=0, color='r', linestyle='--')
plt.tight_layout()
plt.show()
# Visualize the effect of K on model performance
k_range = range(1, 31)
k_mse = []
k_r2 = []
for k in k_range:
    knn = KNeighborsRegressor(n_neighbors=k)
    knn.fit(X_train_scaled, y_train)
    y_pred_k = knn.predict(X_test_scaled)
    k_mse.append(mean_squared_error(y_test, y_pred_k))
    k_r2.append(r2_score(y_test, y_pred_k))
plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
plt.plot(k_range, k_mse)
plt.xlabel('Value of K')
plt.ylabel('MSE')
plt.title('MSE vs. K Value')
plt.subplot(1, 2, 2)
plt.plot(k_range, k_r2)
plt.xlabel('Value of K')
plt.ylabel('R2')
plt.title('R2 vs. K Value')
plt.tight_layout()
plt.show()

```

The results (Screenshot)

Dataset Information:
Number of samples: 20640
Number of features: 8

Feature names:
['MedInc', 'HouseAge', 'AveRooms', 'AveBedrms', 'Population', 'AveOccup', 'Latitude', 'Longitude']

Target description:

First 5 rows of the dataset:

```
MedInc HouseAge AveRooms AveBedrms ... AveOccup Latitude Longitude Target (MedHouseValue)0 8.3252 41.0 6
.984127 1.023810 ... 2.555556 37.88 -122.23 4.5261 8.3014 21.0 6.238137 0.971880 ..
. 2.109842 37.86 -122.22 3.5852 7.2574 52.0 8.288136 1.073446 ... 2.802260 37.85
-122.24 3.5213 5.6431 52.0 5.817352 1.073059 ... 2.547945 37.85 -122.25
3.4134 3.8462 52.0 6.281853 1.081081 ... 2.181467 37.85 -122.25 3.422
[5 rows x 9 columns]
```

Best value of K: 11

Model Evaluation Metrics:

Mean Squared Error (MSE): 0.4186

Root Mean Squared Error (RMSE): 0.6470

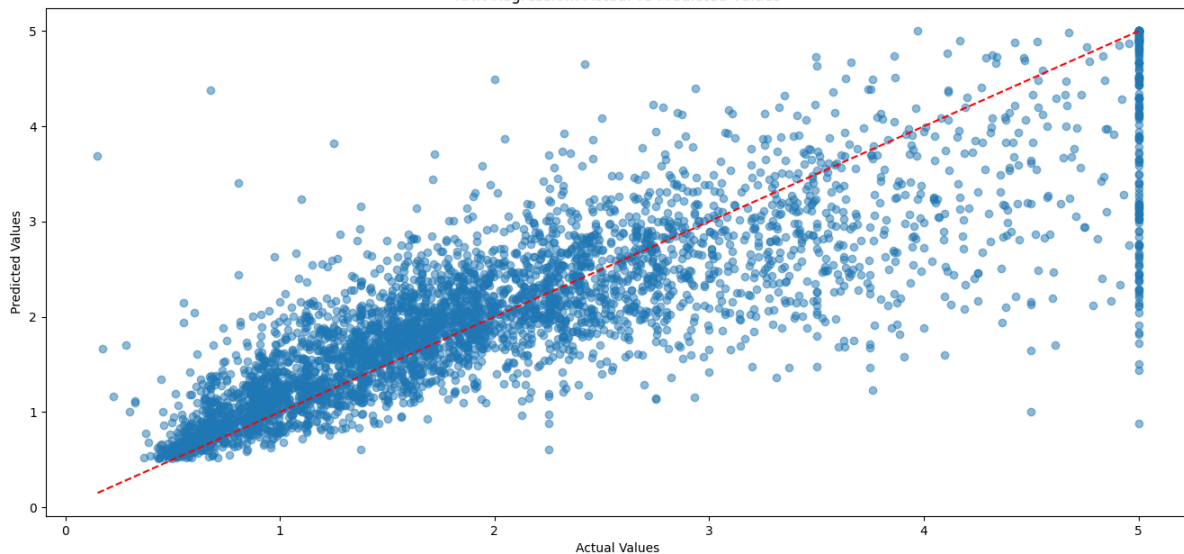
Mean Absolute Error (MAE): 0.4384

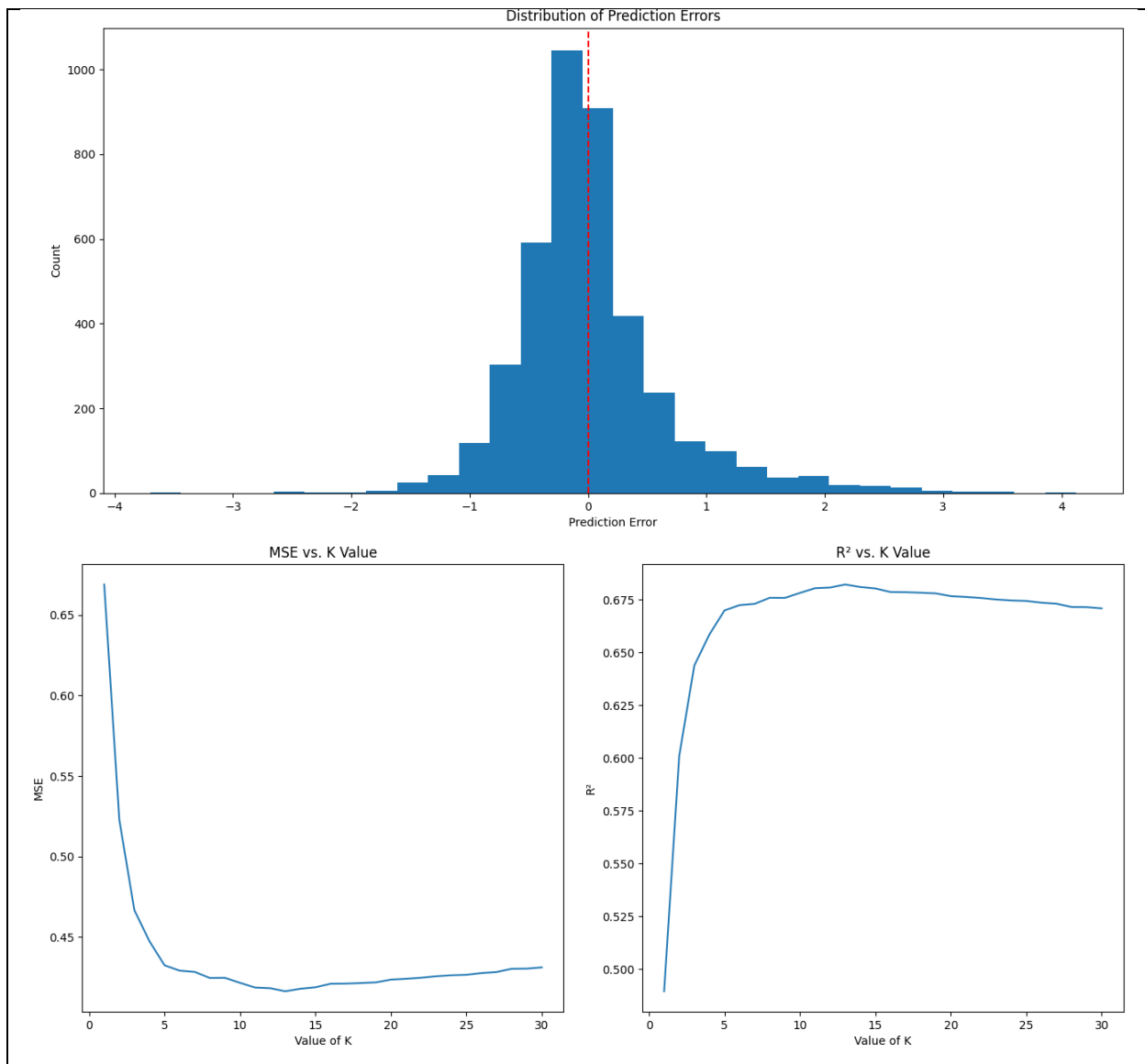
R-squared (R^2): 0.6806

Explained Variance: 0.6808

Mean Absolute Percentage Error (MAPE): 24.4560%

KNN Regression: Actual vs Predicted Values





Conclusion

In conclusion, KNN classifies items according to the classification of the majority among K nearest neighbors within the feature space; for all this simplicity, KNN has indeed proved itself to be powerful for many classification tasks, without an explicit training phase. Key considerations would revolve around choosing the right K value, distance metric, and handling curse of dimensionality; the implementation would include only computing distances between points, sorting neighbors, and taking a vote of the nearest K samples; although it is computationally intensive when predicting, KNN still serves a valuable purpose in smaller datasets and as a baseline classifier to compare against more complex models.